

Documenti autorizzati: due fogli manoscritti A4 fronte retro.

Dovete fare A SCELTA o l'esercizio 4 o l'esercizio 5 MA NON entrambi, in tal caso verrà corretto solo il 4

1 Esercizio 1

<pre> int V1=0; //Variabile condivisa int V2=0; </pre>	<pre> --Process P while(true){ p1: Sezione Non Critica p2: V1=1; p3: if(V2!=0){ p4: goto p2;} p5: else{V2=1;} p6: if(V1!=1 && V2!=1){ p7: goto p2;} p8: Sezione Critica p9: V2=0; } </pre>	<pre> --Process Q while(true){ q1: Sezione Non Critica q2: V1=2; q3: if(V2!=0){ q4: goto q2;} q5: else{V2=2;} q6: if(V1!=2 && V2!=2){ q7: goto q2;} q8: Sezione Critica q9: V2=0; } </pre>
--	--	--

Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1 dove l'istruzione `goto p` manda il programma alla riga p.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati).
2. Spiegare perché questo algoritmo non verifica la proprietà di mutua esclusione.
3. Questo algoritmo verifica la proprietà di assenza di deadlock? Giustificare. (Ricordiamo che c'è deadlock se i due processi rimangono bloccati per sempre fra la Sezione Non Critica e la Sezione Critica).
4. Proporre una modifica delle condizione delle righe p6 e q6 nel modo che l'algoritmo ottenuto verifica la proprietà di mutua esclusione.

2 Esercizio 2

In una città, c'è un bus turistico a due piani, con 10 posti al primo piano e 15 posti al secondo piano. Questo bus ha il seguente comportamento in ciclo:

1. Aspetta alla fermata.
2. Parte per un giro turistico.
3. Aspetta di essere vuoto.

Un turista che vuole prendere questo bus ha il seguente comportamento in ciclo:

1. Se il bus è alla fermata e se c'è posto, prova a salire senno aspetta che queste due condizione siano soddisfatte.
2. Si siede al primo piano se non è pieno, altrimenti si siede al secondo piano.
3. Fa il giro turistico col bus fino alla fine.
4. Scende del bus.

Un turista non può dunque mai salire nel bus se quello non è alla fermata o se è pieno e il bus deve aspettare di essere vuoto prima di potere prendere nuovi clienti. Invece il bus può partire ad ogni istante, anche se è vuoto.

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, prendendo in considerazione che vogliamo un processo per il bus ed un processo per i turisti.

```

--Process Controller
    boolean inCS=false;
    int waiting=0;
    int current=0;
    list Q=emptylist;

    while(true){
c1:   while(waiting==0 || inCS){}
c2:   current=extract(Q);
c3:   inCS=true;
c4:   waiting=waiting-1;
c5:   send(ok)->P(current);
    }

--Gestione dei messaggi del Controller
case (req,i):
    insert(i,Q);
    waiting=waiting+1;
case(out):
    inCS=false;

```

Figura 2: Algoritmo di mutua esclusione con message passing: controllore

```

--Process Pi per i in {0,...,N-1}
    boolean CS=false;

    while(true){
p1:   Sezione Non Critica
p2:   send(req,i)->Controller;
p3:   while(CS==false){}
p4:   Sezione Critica
p5:   send(out)->Controller;
p&   CS=false;
    }

--Gestione dei messaggi di Pi i in {0,...,N-1}
case(ok):
    CS=true;

```

Figura 3: Algoritmo di mutua esclusione con message passing: processi

3 Esercizio 3

Consideriamo l'algoritmo con message passing proposto sulle figure 2 e 3. Per questo algoritmo c'è un processo Controller (il suo comportamento è descritto alla figura 2) e N processi (il loro comportamento è descritto alla figura 3). Il Controller manipola una lista di interi Q che è vuota al inizio e può estrarre un intero di questa lista facendo `extract(Q)` o aggiungere un intero i alla lista facendo `insert(i,Q)`. Lo scopo di questo algoritmo è di garantire la mutua esclusione per gli $N > 1$ processi.

1. Descrivere in qualche righe come funziona questo algoritmo.
2. Questo algoritmo verifica la mutua esclusione? Giustificare.
3. Questo algoritmo verifica l'assenza di deadlock? Giustificare.
4. Con quale condizione sulla lista Q e sulle operazione associate, questo algoritmo verifica l'assenza di starvation? Giustificare.

4 Esercizio 4

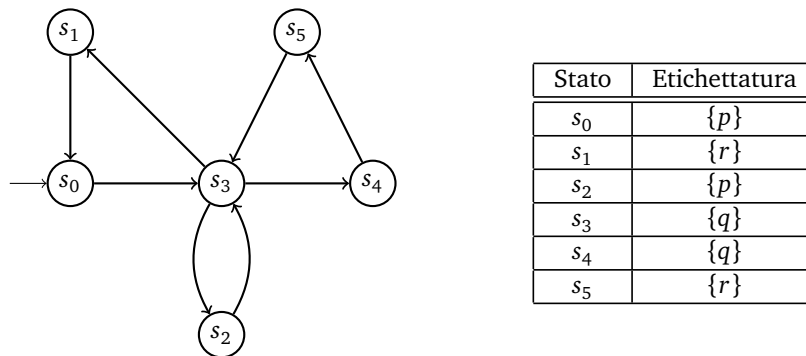


Figura 4: Struttura di Kripke $\mathcal{S}_1$

Consideriamo la struttura di Kripke $\mathcal{S}_1$ rappresentata sulla figura 4 (la tabella a destra indica per ogni stato l'insieme delle proposte atomiche associate) su l'insieme di proposte atomiche $PA = \{p, q, r\}$. Per ciascuna formula ϕ di LTL che seguono, dire (giustificando) se sono soddisfatte dalla struttura (i.e. se abbiamo $\mathcal{S}_1 \models \phi$). 1. $\phi = \mathbf{F}p$ 2. $\phi = \mathbf{F}r$ 3. $\phi = \mathbf{G}Fq$ 4. $\phi = (\mathbf{G}Fr) \Rightarrow (\mathbf{G}Fp)$

5 Esercizio 5

In una compagnia, ci sono tre tipi di sviluppatori che lavorano su un codice sorgente:

1. gli sviluppatori di livello A che leggono solo il codice,
2. gli sviluppatori di livello B che commentano il codice,
3. gli sviluppatori di livello C che modificano il codice.

Ed in più le seguenti regole devono essere rispettate. Quando un sviluppatore di livello C accede al codice, nessun altro sviluppatore può accedere al codice. Quando uno sviluppatore di livello B accede al codice, gli altri sviluppatori di livello B e C non possono accedere ma gli sviluppatori di livello A sì. Più sviluppatori di livello A possono accedere al codice insieme (sia da soli, sia insieme ad uno sviluppatore di livello B). Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, prendendo in considerazione che vogliamo un processo per ogni tipo di sviluppatori.